

Collegium Witelona

Projektowanie i programowanie systemów internetowych I

U Alchemika System

Aplikacja internetowa obiektu agroturystycznego

Sprawozdanie z pracy projektowej

Autorzy: Maciej Bułhak

Małgorzata Bułhak

Hubert Olszak

Przedmiot: Projektowanie i programowanie
systemów internetowych I

Rok akademicki: 2025/2026

Data: 9 czerwca 2026

Stos technologiczny: Django 4.2 (MVT) + PostgreSQL 16
+ Bootstrap 5 + HTMX + Docker

Sprawozdanie przygotowane w technologii L^AT_EX zgodnie z wymaganiami zaliczenia.

Repozytorium: <https://github.com/Bulileusz/u-alchemika-system>

Spis treści

1	Wprowadzenie	2
1.1	Skład zespołu i podział prac	2
1.2	Struktura dokumentu	2
2	Opis funkcjonalny systemu	2
2.1	Cel i charakterystyka systemu	2
2.2	Aktorzy systemu	3
2.3	Mapa funkcji (strony publiczne)	3
2.4	Mapa funkcji (część chroniona)	3
2.5	Scenariusz kluczowy: wysłanie zapytania rezerwacyjnego	3
3	Opis technologiczny	4
3.1	Stos technologiczny	4
3.2	Architektura MVT	4
3.3	Organizacja kodu	5
3.4	Środowisko uruchomieniowe (Docker Compose)	5
3.5	Bezpieczeństwo i konfiguracja	6
4	Wdrożone zagadnienia kwalifikacyjne	6
4.1	Tabela zbiorcza	6
4.2	Framework MVC (#1)	6
4.3	Framework CSS (#2), HTML (#6), CSS (#7)	7
4.4	Baza danych (#3) i ORM (#10)	7
4.5	Cache (#4)	7
4.6	Dependency manager (#5)	8
4.7	JavaScript (#8)	8
4.8	Routing i pretty URLs (#9)	8
4.9	Uwierzytelnianie (#11)	8
4.10	Lokalizacja (#12)	9
4.11	Mailing (#13)	9
4.12	Formularze (#14)	10
4.13	Asynchroniczne interakcje (#15)	10
4.14	RWD (#18)	11
4.15	Logger (#19)	11
4.16	Deployment (#20)	11
4.17	Zagadnienia niewdrożone (#16, #17)	11
5	Instrukcja uruchomienia systemu	11
5.1	Wymagania wstępne	11
5.2	Uruchomienie lokalne (Docker)	12
5.3	Zatrzymanie i reset	12
5.4	Uruchomienie zdalne (środowisko produkcyjne)	12
5.5	Polecenia pomocnicze	13
6	Wnioski projektowe	13

1 Wprowadzenie

Niniejsze sprawozdanie dokumentuje pracę projektową zrealizowaną w ramach kursu *Projektowanie i programowanie systemów internetowych I*. Przedmiotem projektu jest **kompletna aplikacja internetowa** dla obiektu agroturystycznego „U Alchemika”, położonego w Górach Izerskich. Aplikacja udostępnia publiczną witrynę prezentującą ofertę noclegową oraz wewnętrzny panel obsługi zapytań rezerwacyjnych.

Projekt jest realizowany wspólnie z kursem *Projektowanie systemów baz danych* na jednym, współdzielonym repozytorium i kodzie źródłowym. Warstwa danych (model relacyjny, migracje, integralność) została szczegółowo opisana w osobnym dokumencie docs/dokumentacja. Niniejsze sprawozdanie koncentruje się na **warstwie internetowej**: funkcjonalności, technologiach webowych i wdrożonych zagadnieniach kwalifikacyjnych.

1.1 Skład zespołu i podział prac

Osoba	Zakres odpowiedzialności
Maciej Bułhak	Model danych, migracje, integralność, warstwa dostępu do danych (ORM), konfiguracja projektu, konteneryzacja, dokumentacja.
Małgorzata Bułhak	Moduł zapytań rezerwacyjnych (formularze, walidacja, mailing), panel obsługi, logowanie zdarzeń.
Hubert Olszak	Warstwa prezentacji: szablony HTML, style CSS, responsywność (RWD), interaktywność po stronie klienta (JavaScript), dane testowe.

Uwaga: zgodnie z zasadami zaliczenia cały zespół otrzymuje wspólną ocenę, a każdy członek zna całość rozwiązania.

1.2 Struktura dokumentu

Sprawozdanie obejmuje, zgodnie z wymaganiami: opis funkcjonalny systemu (sekcja 2), opis technologiczny (sekcja 3), wyszczególnienie wdrożonych zagadnień kwalifikacyjnych (sekcja 4), instrukcję lokalnego i zdalnego uruchomienia systemu (sekcja 5) oraz wnioski projektowe (sekcja 6).

2 Opis funkcjonalny systemu

2.1 Cel i charakterystyka systemu

System **U Alchemika** to witryna internetowa obiektu noclegowego, której zadaniem jest prezentacja oferty oraz pozyskiwanie zapytań rezerwacyjnych od gości. Przyjęto świadome założenie, że system **nie realizuje pełnej rezerwacji online** – gość wysyła zapytanie, a finalizacja odbywa się poza systemem (kontakt bezpośredni lub przekierowanie do serwisu Booking.com). Dzięki temu zakres funkcjonalny jest skupiony i kompletny w obrębie zdefiniowanych przypadków użycia.

2.2 Aktorzy systemu

Gość (użytkownik anonimowy) – przegląda ofertę (pokoje, galeria, atrakcje, blog) i wysyła zapytanie rezerwacyjne. Nie wymaga konta.

Administrator / obsługa – po uwierzytelnieniu przegląda nadesłane zapytania w panelu wewnętrznym oraz zarządza całością danych poprzez panel Django Admin.

2.3 Mapa funkcji (strony publiczne)

Strona	Adres (URL)	Funkcja
Strona główna	/	Sekcja hero, polecane pokoje, zjawka atrakcji, CTA kontaktu.
Lista pokoi	/pokoje/	Karty aktywnych pokoi z ceną i pojemnością.
Szczegóły pokoju	/pokoje/<slug>/	Galeria, opis, udogodnienia, cena, przejście do zapytania.
Galeria	/galeria/	Zdjęcia obiektu z powiększaniem (lightbox).
Atrakcje	/atrakcje/	Lista atrakcji turystycznych okolicy.
Blog	/blog/	Lista opublikowanych wpisów.
Wpis blogowy	/blog/<slug>/	Treść pojedynczego wpisu.
Kontakt	/kontakt/	Dane kontaktowe obiektu (z cache).
Obiekt	/obiekt/	Polityki obiektu (check-in/out, zwierzęta, płatności).
Zapytanie	/zapytanie/	Formularz zapytania rezerwacyjnego (HTMX).

2.4 Mapa funkcji (część chroniona)

Strona	Adres (URL)	Funkcja
Logowanie	/login/	Uwierzytelnienie obsługi.
Panel zapytań	/panel/zapytania/	Lista zapytań (dostęp tylko po zalogowaniu).
Panel administracyjny	/admin/	Pełny CRUD na wszystkich encjach (Django Admin).
Przełącznik języka	/i18n/setlang/	Zmiana języka interfejsu PL↔DE.

2.5 Scenariusz kluczowy: wysłanie zapytania rezerwacyjnego

Najważniejszy przepływ biznesowy łączy kilka mechanizmów technicznych w jednej interakcji użytkownika:

1. Gość wypełnia formularz zapytania na stronie /zapytanie/.
2. Formularz jest wysyłany **asynchronicznie** (HTMX) – bez przeładowania strony.
3. Serwer waliduje dane (poprawność e-maila, daty, długość pobytu).

4. Po sukcesie: zapytanie zostaje **zapisane w bazie**, na adres gościa wysyłany jest **e-mail potwierdzający**, a zdarzenie trafia do **logów**.
5. Gość widzi sekcję podziękowania (fragment HTML podmieniony przez HTMX).
6. Zapytanie pojawia się w panelu obsługi `/panel/zapytania/`.

Ten pojedynczy scenariusz demonstruje jednocześnie zagadnienia: formularze, asynchroniczność, baza danych, mailing oraz logger.

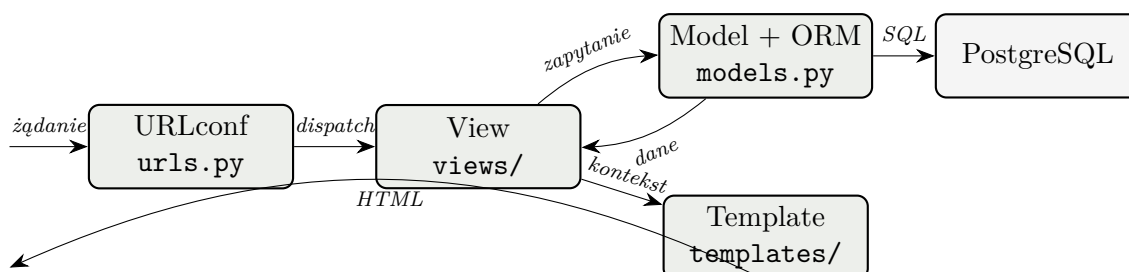
3 Opis technologiczny

3.1 Stos technologiczny

Warstwa	Technologia	Rola w projekcie
Backend / framework	Django 4.2 (Python 3.12)	Logika aplikacji, routing, ORM, szablony.
Baza danych	PostgreSQL 16	Trwałe przechowywanie danych.
Framework CSS	Bootstrap 5.3.3 (CDN)	Siatka, komponenty, responsywność.
Interaktywność	HTMX 1.9.12 + własny JS	Asynchroniczne wysłanie formularza, walidacja klienta.
Poczta (dev)	MailHog	Przechwytywanie maili w środowisku lokalnym.
Cache	LocMemCache (django-redis gotowy)	Buforowanie danych rzadko zmienianych.
Zależności	pip + requirements.txt	Zarządzanie bibliotekami.
Konteneryzacja	Docker Compose	Wdrożenie (web + db + mailhog).

3.2 Architektura MVT

Django realizuje wzorzec **MVT** (Model–View–Template) – odmianę klasycznego MVC, w której rolę kontrolera pełni *View*, a rolę widoku z MVC przejmuje *Template*.



Rysunek 1: Przepływ żądania HTTP w architekturze MVT

- **Model** (`apps/core/models.py`, `apps/content/models.py`) – definicje danych i reguł biznesowych, mapowane przez ORM na tabele.

- **View** (`apps/core/views/`, `apps/content/views.py`) – logika obsługi żądania: pobranie danych, wybór szablonu, walidacja.
- **Template** (`templates/`) – szablony HTML z dziedziczeniem (`base.html`) i partiami dołączanymi przez `{% include %}`.

3.3 Organizacja kodu

Projekt podzielono na dwie aplikacje Django o jasno rozdzielonej odpowiedzialności:

- `apps/core` – pokoje, udogodnienia, zapytania rezerwacyjne, dane obiektu, logi (rdzeń systemu),
- `apps/content` – treści marketingowe: wpisy blogowe i atrakcje.

3.4 Środowisko uruchomieniowe (Docker Compose)

Całość jest skonteneryzowana. Plik `docker-compose.yml` definiuje trzy serwisy:

```

1 services:
2   db:          # PostgreSQL 16 + healthcheck + wolumen
3     postgres_data
4     image: postgres:16
5     healthcheck:
6       test: ["CMD-SHELL", "pg_isready -U postgres -d u_alchemika"]
7   web:         # aplikacja Django
8     build: ./backend
9     ports: ["8000:8000"]
10    depends_on:
11      db:       { condition: service_healthy }
12      mailhog: { condition: service_started }
13  mailhog:     # przechwytywanie poczty (SMTP 1025, UI 8025)
14    image: mailhog/mailhog:latest
15    ports: ["1025:1025", "8025:8025"]
16 volumes:
17   postgres_data:
```

Listing 1: Trzy serwisy aplikacji (fragment `docker-compose.yml`)

Serwis `web` startuje dopiero po przejściu healthchecku bazy. Przy starcie kontener wykonuje automatycznie migracje oraz kompilację tłumaczeń, a następnie uruchamia serwer deweloperski:

```

python manage.py migrate \
  && (python manage.py compilemessages || echo 'brak katalogow
    tłumaczen, pomijam') \
  && python manage.py runserver 0.0.0.0:8000
```

Listing 2: Polecenie startowe kontenera (Dockerfile)

Komunikacja między kontenerami odbywa się po wewnętrznej sieci bridge Dockera – serwis `web` łączy się z bazą po nazwie `db` (wewnętrzny DNS Dockera), a na zewnątrz wystawione są jedynie zmapowane porty.

3.5 Bezpieczeństwo i konfiguracja

- Dane wrażliwe (klucz, dostęp do bazy) wczytywane są ze zmiennych środowiskowych przez `python-decouple` – brak sekretów w kodzie.
- Ochrona CSRF (token w każdym formularzu, `CsrfViewMiddleware`).
- Hashowanie haseł i walidatory siły hasła (`AUTH_PASSWORD_VALIDATORS`).
- Automatyczne escapowanie zmiennych w szablonach (ochrona przed XSS).

4 Wdrożone zagadnienia kwalifikacyjne

Spośród 20 zagadnień kwalifikacyjnych wdrożono **18**, co zgodnie z tabelą oceniania ustala pułap oceny za pracę projektową na **5,0**. Poniższa tabela podsumowuje stan realizacji, a dalsze podsekcje opisują każde zagadnienie wraz z odniesieniem do kodu.

4.1 Tabela zbiorcza

#	Zagadnienie	Status	Dowód w kodzie
1	framework MVC	✓	Django 4.2 (MVT)
2	framework CSS	✓	Bootstrap 5.3 (CDN)
3	baza danych	✓	PostgreSQL 16 (kontener db)
4	cache	✓	<code>cache.get/set</code> , <code>@cache_page</code>
5	dependency manager	✓	<code>pip</code> + <code>requirements.txt</code>
6	HTML	✓	szablony + dziedziczenie <code>base.html</code>
7	CSS	✓	własny <code>style.css</code> + Bootstrap
8	JavaScript	✓	<code>inquiry.js</code> , lightbox galerii
9	routing	✓	<code>urls.py</code> , pretty URLs, <code><slug></code>
10	ORM	✓	<code>modele</code> , <code>prefetch/select_related</code>
11	uwierzytelnianie	✓	<code>LoginView</code> , <code>@login_required</code>
12	lokalizacja	✓	przełącznik PL/DE, <code>{% trans %}</code>
13	mailing	✓	<code>send_mail</code> → MailHog
14	formularze	✓	<code>InquiryForm(ModelForm)</code> + walidacje
15	asynchroniczne interakcje	✓	HTMX (partiale HTML)
16	konsumpcja API	—	niewdrożone (świadoma luka)
17	publikacja API	—	niewdrożone (świadoma luka)
18	RWD	✓	<code>viewport</code> + siatka Bootstrap + media queries
19	logger	✓	<code>LOGGING</code> + <code>logger.*</code>
20	deployment	✓	Docker Compose (web+db+mailhog)
Suma zaliczonych:			18 / 20 ⇒ pułap 5,0

4.2 Framework MVC (#1)

Backend oparto na **Django 4.2** w architekturze MVT (omówionej w sekcji 3). Przepływ: `urls.py` → widok → ORM → szablon → HTML.

4.3 Framework CSS (#2), HTML (#6), CSS (#7)

Warstwę prezentacji oparto na **Bootstrap 5.3** (siatka responsywna, komponenty: karty, przyciski, alerty, tabela, klasy walidacji formularza) uzupełnionym o własny arkusz `static/css/style.css` nadający charakter marce (navbar z logo, sekcja hero, karty pokoi, galeria). Szablony HTML wykorzystują **dziedziczenie**: wspólny `base.html` zawiera nawigację i stopkę, a podstrony nadpisują blok treści. Powtarzalne fragmenty (formularz, polecane pokoje) wydzielono do partiali dołączanych przez `{% include %}` – zero duplikacji.

4.4 Baza danych (#3) i ORM (#10)

Dane przechowuje **PostgreSQL 16** w osobnym kontenerze. Dostęp realizuje wyłącznie **Django ORM** – bez ręcznego SQL. Zastosowano optymalizację zapytań przeciw problemowi N+1:

```

1 # Lista pokoi: jedno dodatkowe zapytanie zamiast N (relacja 1:N)
2 Room.objects.filter(is_active=True).prefetch_related("images")
3
4 # Szczegóły pokoju: zdjęcia + udogodnienia (1:N oraz M:N)
5 Room.objects.prefetch_related("images", "amenities")
6
7 # Panel zapytań: JOIN z pokojem jednym zapytaniem (FK "w przód")
8 Inquiry.objects.select_related("room").order_by("-created_at")

```

Listing 3: Optymalizacja zapytań ORM (views/rooms.py, views/inquiries.py)

Model relacyjny (encje, klucze obce, relacja M:N przez tabelę pośrednią, indeksy i migracje) opisano szczegółowo w dokumentacji bazy danych.

4.5 Cache (#4)

Skonfigurowano backend `LocMemCache`. Buforowanie zastosowano w dwóch wariantach:

```

1 # 1. Cache niskopoziomowy - dane kontaktowe (rzadko sie zmieniaja
2   )
3 prop = cache.get("property_info_contact")
4 if prop is None:
5     prop = PropertyInfo.objects.first()
6     cache.set("property_info_contact", prop, timeout=60 * 15) #
7       15 min
8
9 # 2. Cache calej strony - dekorator
10 @cache_page(60 * 30) # strona /obiekt/ buforowana na 30 minut
11 def property_info(request): ...

```

Listing 4: Dwa wzorce cache (views/inquiries.py)

W zależnościach znajduje się `django-redis` jako gotowa alternatywa (drop-in) umożliwiająca współdzielenie cache między procesami w środowisku produkcyjnym.

4.6 Dependency manager (#5)

Zależności deklaruje `backend/requirements.txt` z kontrolą wersji (`Django>=4.2,<5.0`, `psycpg2-binary`, `python-decouple`, `django-redis==5.4.0`). Instalacja (`pip install -r requirements.txt`) odbywa się automatycznie podczas budowy obrazu Docker, co gwarantuje powtarzalne i identyczne środowisko u każdego członka zespołu i na serwerze.

4.7 JavaScript (#8)

Skrypt `static/core/js/inquiry.js` wzbogaca formularz zapytania: licznik znaków pola wiadomości, walidacja dat po stronie klienta (minimalna data = dziś, data wyjazdu po przyjeździe) oraz ponowne podpięcie zdarzeń po asynchronicznej podmianie formularza przez HTMX (zdarzenie `htmx:afterSwap`). Galeria zdjęć obsługuje powiększanie obrazów (lightbox) z zamykaniem klawiszem Esc.

4.8 Routing i pretty URLs (#9)

Routing jest hierarchiczny – główny `config/urls.py` deleguje przez `include()` do plików `urls.py` poszczególnych aplikacji. Zastosowano **pretty URLs** z konwerterem `<slug:slug>` oraz trasy **nazwane** z przestrzeniami nazw (`core:`, `content:`).

```

1 app_name = "core"
2 urlpatterns = [
3     path("",                                rooms.index,          name="index")
4     ,
5     path("pokoje/",                          rooms.room_list,       name="
6         room_list"),
7     path("pokoje/<slug:slug>/",              rooms.room_detail,    name="
8         room_detail"),
9     path("galeria/",                          rooms.gallery,         name="gallery
10    ),
11    path("zapytanie/",                        inquiry_create,        name="inquiry
12        -create"),
13    path("kontakt/",                          contact,               name="contact
14        "),
15    path("panel/zapytania/",                  inquiry_list,          name="inquiry
16        -list"),
17 ]

```

Listing 5: Routing aplikacji core (`apps/core/urls.py`)

Dzięki nazwanym trasom szablony nie zawierają adresów na sztywno (np. `{% url 'core:room_detail' room.slug %}`), co umożliwia zmianę ścieżki w jednym miejscu.

4.9 Uwierzytelnianie (#11)

Wykorzystano wbudowany system Django: `LoginView/LogoutView`, model `auth.User`, hashowanie haseł i walidatory ich siły. Panel zapytań jest chroniony dekoratorem `@login_required` – próba wejścia bez sesji skutkuje przekierowaniem na `/login/` z parametrem `next`.

```

1 @login_required(login_url="login")
2 def inquiry_list(request):

```

```

3     inquiries = Inquiry.objects.select_related("room").order_by("
        -created_at")
4     return render(request, "core/inquiry_list.html", {"inquiries"
        : inquiries})

```

Listing 6: Ochrona panelu obsługi (views/inquiries.py)

4.10 Lokalizacja (#12)

Zaimplementowano pełne i18n z przełącznikiem **PL/DE** w pasku nawigacji. Konfiguracja obejmuje `LANGUAGES=[pl, de]`, `LocaleMiddleware` oraz `LOCALE_PATHS`. Przełączanie języka realizuje wbudowany widok `set_language` (wybór zapisywany w cookie `django_language`), bez prefiksów w URL – adresy pozostają niezmienione.

```

<form action="{% url 'set_language' %}" method="post" class="lang
-switch">
    {% csrf_token %}
    <input name="next" type="hidden" value="{ request.path }">
    <button name="language" value="pl" class="lang-btn ...">PL/<
        button>
    <button name="language" value="de" class="lang-btn ...">DE/<
        button>
</form>

```

Listing 7: Przełącznik języka w nawigacji (base.html)

Teksty interfejsu oznaczono `{% trans %}/{% blocktrans %}` w szablonach oraz `gettext/gettext_lazy` w kodzie Pythona (etykiety formularza, komunikaty, walidacje). Tłumaczenia niemieckie przechowuje katalog `backend/locale/de/LC_MESSAGES/django.po`, kompilowany do binarnego `.mo` przy starcie kontenera. Zakresem tłumaczenia objęto interfejs aplikacji; treść redakcyjna z bazy (opisy pokoi, wpisy) pozostaje w języku polskim.

4.11 Mailing (#13)

Po zapisaniu zapytania system wysła e-mail potwierdzający na adres gościa. W środowisku lokalnym SMTP skierowano do **MailHog** (kontener przechwytuje pocztę, podgląd pod `http://localhost:8025`). Treść maila pochodzi z szablonu, a ewentualny błąd wysyłki jest przechwytywany i logowany, nie przerywając zapisu zapytania.

```

1 def _send_inquiry_confirmation(inquiry):
2     body = render_to_string("core/emails/inquiry_confirmation.txt
        ", {"inquiry": inquiry})
3     try:
4         send_mail(subject=..., message=body, from_email="
            noreply@u-alchemika.pl",
5                 recipient_list=[inquiry.email], fail_silently=
            False)
6         logger.info("E-mail wyslany na %s", inquiry.email)
7     except Exception as exc:
8         logger.error("Blad wysylki e-maila dla zapytania #%d: %s"
            , inquiry.pk, exc)

```

Listing 8: Wysyłka potwierdzenia (views/inquiries.py)

4.12 Formularze (#14)

`InquiryForm` dziedziczy po `forms.ModelForm` – pola generowane są z modelu `Inquiry`, bez dublowania definicji. Walidacja działa na wielu poziomach: typy pól (np. `EmailField`), reguły formularza oraz reguła modelu.

```

1 def clean_date_from(self):
2     date_from = self.cleaned_data.get("date_from")
3     if date_from and date_from < timezone.localtime():
4         raise forms.ValidationError(_("Data przyjazdu nie moze
5             byc w przeszlosci."))
6     return date_from
7
8 def clean(self):
9     cleaned = super().clean()
10    date_from, date_to = cleaned.get("date_from"), cleaned.get("
11        date_to")
12    if date_from and date_to:
13        if date_to <= date_from:
14            self.add_error("date_to", _("Data wyjazdu musi byc
15                pozniejsza..."))
16        elif (date_to - date_from).days > 30:
17            self.add_error("date_to", _("Pobyt nie moze trwac
18                dluzej niz 30 dni."))
19    return cleaned

```

Listing 9: Walidacja wielopoziomowa (forms.py)

Każdy formularz zabezpieczony jest tokenem CSRF (`{% csrf_token %}`).

4.13 Asynchroniczne interakcje (#15)

Wysłanie formularza zapytania jest asynchroniczne dzięki **HTMX**. Formularz deklaruje atrybuty `hx-post`, `hx-target`, `hx-swap`. Widok rozpoznaje żądanie po nagłówku `HX-Request` i zwraca **fragment HTML** (nie JSON): przy sukcesie sekcję podziękowania, przy błędzie ten sam formularz z komunikatami przy polach.

```

1 if form.is_valid():
2     inquiry = form.save()
3     _send_inquiry_confirmation(inquiry)
4     if request.headers.get("HX-Request"):
5         return render(request, "core/_inquiry_success.html", {"
6             inquiry": inquiry})
7     ...
8 else:
9     if request.headers.get("HX-Request"):
10        return render(request, "core/_inquiry_form.html", {"form":
11            form})

```

Listing 10: Obsługa żądania HTMX (views/inquiries.py)

4.14 RWD (#18)

Responsywność zapewnia meta `viewport`, responsywna siatka Bootstrap (karty pokoi: 3 w rzędzie → 1 na telefonie) oraz własne media queries w `style.css`. Pasek nawigacji wraz z przełącznikiem języka zawija się na węższych ekranach, a przełącznik PL/DE ładuje na osobnej, wyśrodkowanej linii.

4.15 Logger (#19)

Skonfigurowano `LOGGING` (handler konsolowy, formatter, logger `apps` na poziomie `INFO`). Rejestrowane są kluczowe zdarzenia biznesowe: nowe zapytanie (`info`), nieprawidłowy formularz (`warning`), nieudana wysyłka maila (`error`) oraz dostęp do panelu zapytań.

```
1 logger.info("Nowe zapytanie #%d od %s <%s> na pokój '%s' (%s-%s)"
2             ,
3             inquiry.pk, inquiry.full_name, inquiry.email,
              inquiry.room, inquiry.date_from, inquiry.date_to)
```

Listing 11: Logowanie zdarzeń (views/inquiries.py)

4.16 Deployment (#20)

Wdrożenie realizuje **Docker Compose** (serwisy `web`, `db`, `mailhog`). Uruchomienie sprowadza się do jednej komendy `docker compose up -build`; migracje i kompilacja tłumaczeń wykonują się automatycznie. Szczegóły w sekcji 5.

4.17 Zagadnienia niewdrożone (#16, #17)

Świadomie pominięto **konsumpcję API** (#16) oraz **publikację API** (#17). Przy 18 zaliczonych zagadnieniach pułap oceny wynosi już 5,0, więc dalsze rozszerzanie zakresu nie wpływa na ocenę. Naturalnym kierunkiem rozbudowy byłoby: dla #16 – pobranie mapy (np. Leaflet) lub danych pogodowych na stronie atrakcji; dla #17 – prosty endpoint `/api/pokoje/` zwracający listę pokoi jako JSON (`JsonResponse`, nawet bez Django REST Framework).

5 Instrukcja uruchomienia systemu

5.1 Wymagania wstępne

- Docker Engine ≥ 24 oraz Docker Compose V2,
- Git,
- wolne porty: 8000 (web), 5432 (db), 8025/1025 (MailHog).

5.2 Uruchomienie lokalne (Docker)

```
# 1. Klonowanie repozytorium
git clone https://github.com/Bulileusz/u-alchemika-system.git
cd u-alchemika-system

# 2. Uruchomienie kontenerow (migracje i kompilacja tlumaczen -
    automatycznie)
docker compose up --build

# 3. (Nowe okno) Zaladowanie danych testowych
docker compose exec web python manage.py loaddata seed

# 4. Utworzenie konta administratora (do panelu i /admin/)
docker compose exec web python manage.py createsuperuser
```

Listing 12: Pierwsze uruchomienie

Po uruchomieniu dostępne są:

- witryna publiczna: <http://localhost:8000/>,
- panel administracyjny: <http://localhost:8000/admin/>,
- skrzynka MailHog (podgląd maili): <http://localhost:8025/>.

5.3 Zatrzymanie i reset

```
docker compose down          # zatrzymanie (dane zachowane)
docker compose down -v      # zatrzymanie + reset bazy (usuwa
    wolumen)
```

Listing 13: Zatrzymanie kontenerów

5.4 Uruchomienie zdalne (środowisko produkcyjne)

Konteneryzacja sprawia, że wdrożenie na serwer (VPS / chmura) sprowadza się do uruchomienia tego samego `docker compose` na maszynie zdalnej. Względem konfiguracji deweloperskiej należy wprowadzić następujące zmiany produkcyjne:

Element	Zmiana produkcyjna
DEBUG	ustawić na <code>False</code> .
DJANGO_SECRET_KEY	wygenerować długi, losowy klucz i podać przez zmienną środowiskową.
DJANGO_ALLOWED_HOSTS	wpisać domenę/IP serwera.
Serwer WWW	zastąpić <code>runserver</code> serwerem produkcyjnym (np. Gunicorn) za reverse proxy (np. Nginx).
Pliki statyczne	wykonać <code>collectstatic</code> i serwować je przez Nginx (lub WhiteNoise).
Poczta	zastąpić MailHog realnym serwerem SMTP (dane z env).

Element	Zmiana produkcyjna
Baza danych	użyć trwałego wolumenu lub zarządzanej usługi PostgreSQL; ustawić silne hasło.
HTTPS	skonfigurować certyfikat TLS (np. Let's Encrypt) na reverse proxy.

Przykładowa sekwencja wdrożenia na serwerze zdalnym:

```
# Na serwerze (z zainstalowanym Dockerem):
git clone https://github.com/Bulileusz/u-alchemika-system.git
cd u-alchemika-system

# Ustawienie zmiennych produkcyjnych (przykład)
export DJANGO_SECRET_KEY="<dlugi-losowy-klucz>"
export DEBUG=False
export DJANGO_ALLOWED_HOSTS="u-alchemika.example.com"

# Uruchomienie w tle
docker compose up --build -d

# Zebranie plików statycznych i konto administratora
docker compose exec web python manage.py collectstatic --noinput
docker compose exec web python manage.py createsuperuser
```

Listing 14: Wdrożenie na serwerze zdalnym

*Uwaga: bieżąca wersja repozytorium jest skonfigurowana pod środowisko deweloperskie (serwer **runserver**, MailHog, domyślne sekrety). Powyższa instrukcja opisuje kroki konieczne do bezpiecznego wdrożenia produkcyjnego.*

5.5 Polecenia pomocnicze

```
# Stan migracji / rollback
docker compose exec web python manage.py showmigrations
docker compose exec web python manage.py migrate core 0002

# Testy
docker compose exec web python manage.py test

# Podgląd tabel w bazie
docker compose exec db psql -U postgres -d u_alchemika -c "\dt"
```

Listing 15: Migracje, testy, podgląd bazy

6 Wnioski projektowe

1. **Framework przyspiesza i porządkuje pracę.** Wybór Django pozwolił skupić się na logice biznesowej obiektu noclegowego zamiast na „hydraulice” (routing, ORM, formularze, panel admina, bezpieczeństwo). Mechanizmy gotowe z pudełka (CSRF,

hashowanie haseł, escapowanie XSS) podniosły bezpieczeństwo bez dodatkowego nakładu.

2. **Konteneryzacja eliminuje problem „u mnie działa”.** Docker Compose zapewnił identyczne, powtarzalne środowisko dla całego zespołu i uprościł ścieżkę do wdrożenia zdalnego – ten sam plik kompozycji działa lokalnie i na serwerze.
3. **Dobór technologii proporcjonalny do skali.** Zamiast ciężkiego frameworka frontendowego (React) wybrano HTMX, który dokłada asynchroniczność do szablonów serwerowych przy minimalnym koszcie. Dla pojedynczego interaktywnego formularza było to rozwiązanie adekwatne.
4. **Świadome ograniczenie zakresu.** Rezygnacja z pełnej rezerwacji online oraz z tłumaczenia treści redakcyjnej pozwoliła dostarczyć kompletny, dopracowany zakres zamiast wielu niedokończonych funkcji. Analogicznie, przy 18 zaliczonych zagadnieniach (pułap 5,0) świadomie pominięto zagadnienia API, które nie wnoszą wartości do tego konkretnego systemu.
5. **Optymalizacja wydajności ma znaczenie już na małej skali.** Zastosowanie `prefetch_related/select_related` oraz cache dla danych rzadko zmienianych pokazało, jak prostymi środkami ogranicza się liczbę zapytań do bazy.
6. **Kierunki rozbudowy.** Naturalne rozszerzenia to: wdrożenie produkcyjne za reverse proxy z HTTPS, dodanie konsumpcji i publikacji API (#16, #17), pełne tłumaczenie treści (`django-modeltranslation`) oraz podpięcie współdzielonego cache Redis (zależność jest już dołączona).

Metryka projektu	Wartość
Aplikacje Django	2 (core, content)
Strony publiczne	10
Zaliczone zagadnienia	18 / 20
Pułap oceny za projekt	5,0
Serwisy kontenerów	3 (web, db, mailhog)
Obsługiwane języki	2 (PL, DE)